



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

Precondiciones:

✓ *Ud. ha asistido a clase de teoría y ha leído los apuntes de la cátedra.*

1. Dado el siguiente programa indicar su salida:

```
with ada.integer_text_io; use ada.integer_text_io;
procedure uno is
type pinte is access integer;
a: pinte;
b: integer;
procedure cuacua (x: pinte; y: in out integer) is
begin
  x.all := x.all * x.all;
  y := y * y;
end cuacua;
begin
  a:= new integer;
  a.all := 2;
  b := 3;
  cuacua (a, b);
  put (a.all);
  put (b);
end uno;
```

2. Explicar que hace el siguiente programa:

```
with Ada.Unchecked_Deallocation;
procedure MiEjemplo is
type MiTipo is array(1..1000) of integer;
type PuntMiTipo is access MiTipo;
procedure free is new Ada.Unchecked_Deallocation(MiTipo, PuntMiTipo);
MiVariable: puntMiTipo;
begin
  MiVariable:= new MiTipo;
  MiVariable (1):= 12;
  MiVariable (2):=34;
  MiVariable (852):=25;
  free (MiVariable);
end MiEjemplo;
```

3. Hacer el seguimiento en papel de las variables del siguiente programa, ¿qué observa en particular? ¿podría mejorarse? ¿cómo?



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

```
with Gnat.IO; use Gnat.IO;
with Ada.Unchecked_Deallocation;
procedure dos_1 is
  subtype str5 is string (1..5);
  type PuntStr is access str5;
  procedure free is new Ada.Unchecked_Deallocation (str5, PuntStr);
  pa, pb: PuntStr;
begin
  pa:= new str5("Hola ");
  pb:= new str5(1..5=>' ');
  pa:= new str5(1..5=>' ');
  pb:= pa;
  put_line (pa.all);
  pb.all:="Adios";
  put_line (pa.all);
  free (pb);
  free (pa);
end dos_1;
```

4. Suponer las siguientes declaraciones:

```
type ptrfloat is access float;
type ptrint is access integer;
X: integer;
P1, P2: ptrint;
Q1, Q2: ptrfloat;
```

¿Qué error (si es que hay alguno) hay en cada una de las siguientes instrucciones?

- a) put (P1); -
- b) get (P1);
- c) P1:= Q1;
- d) x:= new integer;
- e) if P1 = null then Q1:= Q2;
 end if;
- f) begin
 P1:= 17;
 P1:= new integer;
end;



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

5. Dada la siguiente declaración, representar gráficamente las distintas operaciones sobre las variables de tipo puntero e indicar el contenido de las variables asociadas.

```
type Ptr_int is access integer;  
type Ptr_char is access character;  
type coche;  
type Ptr_coche is access coche;  
type coche is record  
    año_fab: integer;  
    marca: string (1..10);  
    patente:string (1..6);  
end record;  
  
p1, p2: Ptr_int;  
k1, k2: Ptr_char;  
c1, c2: Ptr_coche;  
a: coche;
```

- | | |
|--|--|
| a) p1 := new integer; p2 := new integer; | b) p1.all:= 20; |
| c) p2:= p1; | d) p2.all:= 33 + p1.all; |
| e) put (p2.all - 5); | f) k1 := new character; k2 := new character; |
| g) k1.all:= 'A'; k2.all:= 'Z'; | h) k2.all:=k1.all; |
| i) k1:= k2; | j) c1 := new coche; c2 := new coche; |
| k) get (a.año_fab);get (a.marca); get (a.patente); | l) c1.all:= a; |
| m) c1.marca:= "Citroen"; | n) c2.patente := a.patente; |
| o) c2 := c1; | |

6. Indique la frase verdadera. Si existen varias alternativas verdaderas, márquelas todas.

6.1. Teniendo en cuenta que pDos y pUno son punteros a integer, ¿es correcta la siguiente secuencia de instrucciones para duplicar el valor numérico apuntado por pUno?

(I) pDos := pUno;

(II) pDos.all := pDos.all *2;

- a) No, porque es imprescindible la instrucción pDos:= **new** integer antes de utilizar el puntero pDos.
- b) Sí, porque la instrucción (II) duplica el valor numérico.
- c) No, porque nosotros queremos duplicar el valor apuntado por pUno y la instrucción (II) duplica el valor numérico apuntado por pDos, no el de pUno.
- d) Sí, porque la instrucción (II) duplica la dirección de memoria donde está ubicado pDos, duplicando así el valor al que apunta.



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

6.2. Después de definir **type** pun **is access** integer, declarar vPun: pun y ejecutar la operación vPun := **new** integer, la instrucción vPun.all := 3,

- a) Devuelve la dirección de memoria donde está ubicado el valor numérico 3.
- b) Ubica el valor numérico 3 en la dirección de memoria contenida en la variable vPun
- c) Ubica el valor numérico 3 en la dirección de memoria donde está ubicada la variable vPun ,
- d) Reserva memoria para almacenar el número 3.

6.3. Dado el siguiente programa:

-- precondition: Unchecked_Deallocation fue instanciado para crear el procedimiento free

procedure ejercicio **is**

type pun **is access** integer;

 b: integer;

 a, c, d: pun;

begin

 a := **new** integer;

 a.all:=3;

 b:=5;

 c := **new** integer;

 d := c;

 d.all := a.all;

 c:=a;

 c.all := b;

 put (a.all); put (' '); put(b); put (' '); put (c.all); put (' '); put (d.all);

 new_line;

 c:= d;

 c.all := 2;

 put (a.all); put (' '); put (b); put (' '); put (c.all); put (' '); put (d.all);

 free (d);

end ejercicio;

a) Mostrará en pantalla:

5 5 5 3

5 5 2 2

b) Habrá un error de ejecución al ejecutar la instrucción free (d)

c) Mostrará en pantalla

3 5 5 3

5 5 2 3

d) Mostrará en pantalla:



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

3 5 5 3

3 5 2 2

7. Suponer las siguientes declaraciones:

```
type puntInt is access integer;  
type Nodonumero;  
type PtNumero is access Nodonumero;  
type NodoNumero is record  
    Datos: integer;  
    Sig: PtNumero;  
end record;
```

```
P1, P2: PtNumero;  
P3: puntInt;
```

Suponer que se han ejecutado las siguientes instrucciones

```
P1:= new Nodonumero;  
P2:= new Nodonumero;  
P3:= new integer;
```

Decir que resultado se visualizará para cada uno de los siguientes fragmentos de programa (en caso de que haya algún error indíquelo).

- a) P1.Datos := 123;
P2.Datos := 456;
P1.Sig := P2;
put (P1.Datos);
put (P1.Sig.Datos);
- b) P1.Datos:= 12;
P2.Datos:= 34;
P1 := P2;
put (P1.Datos);
put (P2.Datos);
- c) P1.Datos:= 12;
P2.Datos:= 34;
P3.Datos:= 34;
P1.Sig := P2;
P2.Sig := P3;
put (P1.Datos);
put (P2.Datos);
put (P3.Datos);



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

```
d) P1.Datos:= 12;  
   P2.Datos:= 34;  
   P1 := P2;  
   P2.Sig := P1;  
   put (P1.Datos);  
   put (P2.Datos);  
   put (P1.Sig.Datos);  
   put (P2.Sig.Datos);
```

8. Dada la siguiente declaración:

```
type TipoLista;  
type TipoPuntero is access TipoLista;      {1}  
type TipoLista is record                    {2}  
    Info: AlgunTipo;  
    Siguiente: TipoPuntero;  
end record;  
UnaLista, OtraLista: TipoPuntero;
```

- a) Explique el significado de {1}.
- b) En la declaración {2}, ¿cuál es el significado del campo *Siguiente*?
- c) Explique cuáles de las siguientes expresiones son válidas:

UnaLista := OtraLista		UnaLista.Siguiente := OtraLista.Info	
UnaLista.Info := OtraLista		UnaLista.Siguiente OtraLista.Siguiente	:=
UnaLista.all		UnaLista.Siguiente := OtraLista.all	

9. Dadas las siguientes declaraciones de tipo, indicar con √ cuáles asignaciones son válidas o no.

```
type TipoNodo;  
type TipoPuntero is access TipoNodo  
type OTipoNodo;  
type OtipoPuntero is access OTipoNodo  
type TipoNodo is record  
    Info: AlgunTipo;  
    Siguiente: TipoPuntero;  
end record;  
type OtipoNodo is record  
    Info: AlgunTipo;  
    Siguiente: OtipoPuntero;  
end record;
```

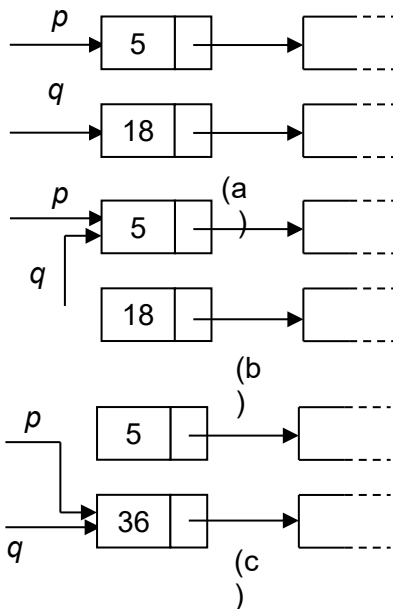


ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

```
Lista1, Lista2 : TipoPuntero;  
Olista1, Olista2 : OtipoPuntero;  
begin
```

```
.....  
Lista1 := Lista2.Siguiente  
Lista2 := Olista1;  
Olista2 := Lista1.all;  
Olista1.Siguiente := Lista2;  
Lista2.info := Olista1.info;
```


10. Dadas las variables puntero p y q , indicar las expresiones correspondientes para que los nodos contengan lo que se muestra en el gráfico:



Expresión

Campo Info de

(a)..... p q

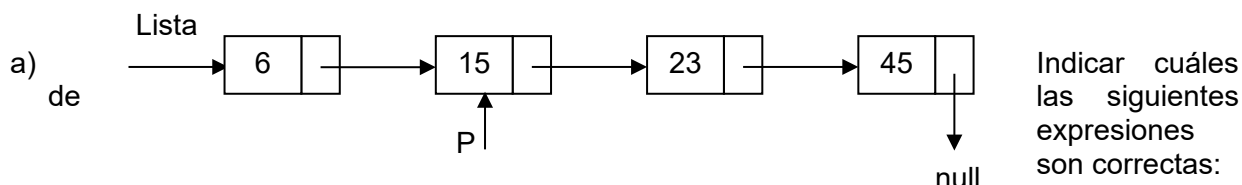
(b)..... p q

(c)..... p q



ALGORÍTMICA Y PROGRAMACIÓN II
TRABAJO PRÁCTICO Nº 3 – ASIGNACIÓN DINÁMICA DE MEMORIA

11. Dada la siguiente Lista:



<i>Lista.siguiete = 6</i>		<i>Lista.info = 6</i>	
<i>Lista.siguiete.siguiete = 6</i>		<i>P.siguiete.info = 23</i>	
<i>P.siguiete.siguiete = null</i>		<i>P.siguiete.info = 6</i>	
<i>P /= null</i>		<i>Lista.siguiete.siguiete = null</i>	
<i>Lista.siguiete = P</i>		<i>Lista.siguiete.all = P.all</i>	

12. Dado el siguiente código, analizarlo (en papel) e indicar cual es su efecto:

procedure algo (lista: tipopuntero; z: **out** float) **is**

ant, ptr: tipopuntero;

w: integer:= 0;

e: float;

begin

z:= 0.0;

ptr:= lista;

while ptr /= null **loop**

w:= w+1;

e:= ptr.info;

ant:= ptr;

ptr:= ptr.siguiete;

free (ant);

z:= z + e;

end loop;

z:= z / float (w);

end algo;

Poscondiciones:



Ud. ha resuelto todos los ejercicios propuestos y maneja variables de tipo puntero para la asignación dinámica de memoria.